Visão geral

Neste projeto, cada grupo irá projetar, treinar e avaliar um *Small Language Model* (SLM). Sugere-se que o modelo tenha entre 50 milhões e 200 milhões de parâmetros treináveis. O objetivo é reproduzir, em escala reduzida, o *pipeline* completo de desenvolvimento de LLMs: desde o *design* da arquitetura e o pré-treino em larga escala, passando pelo *mid-training* e pelo *supervised fine-tuning* (SFT), até a avaliação quantitativa e uma demonstração em modo *chat*.

O projeto é deliberadamente aberto. Não há uma única solução correta: diferentes escolhas de arquitetura, dados e hiperparâmetros levarão a modelos com características distintas. Explorar e justificar essas escolhas é parte central da avaliação.

O desenvolvimento do SLM segue quatro etapas sequenciais:

$$\text{Pré-treino} \longrightarrow \textit{Mid-training} \longrightarrow \text{SFT} \longrightarrow \text{Avaliação \& Demo}$$

A entrega é única e cobre todas as etapas.

Etapa 1: pré-treino

Objetivo

O pré-treino tem como objetivo aprender representações gerais de linguagem a partir de um grande *corpus* de texto não rotulado. O modelo deve aprender a prever o próximo *token* de forma autorregressiva (*next-token prediction*).

Projeto da arquitetura

Cada grupo deve projetar a arquitetura do seu SLM. O ponto de partida é um *transformer decoder-only* (estilo GPT), mas os grupos estão encorajados a explorar e justificar variantes. A seguir estão os principais componentes a considerar:

Componente	**Exemplos e alternativas a explorar**
Positional Encoding	RoPE, ALiBi, aprendido, sinusoidal
Mecanismo de atenção	MHA, GQA (*Grouped-Query Attention*), MQA, *Flash Attention*
Função de ativação	GELU, SwiGLU, ReLU, SiLU
Normalização	LayerNorm (*pre-norm*), RMSNorm, *post-norm*
Otimizador	AdamW, Adam, Muon; *schedule*: cosseno, WSD
Tokenizer	BPE (GPT-2), BPE com vocabulário maior, SentencePiece, treinado do zero

A contagem de parâmetros treináveis deve se encaixar no intervalo de 50M a 200M. Grupos que extrapolarem esse intervalo deverão justificar tecnicamente a decisão.

Conjunto de dados e *scaling laws*

A escolha do *corpus* de pré-treino e da quantidade de *tokens* é responsabilidade do grupo. Recomenda-se o uso das *Chinchilla Scaling Laws* como guia: o número ótimo de *tokens* de treinamento é aproximadamente $20 \times N$, onde N é o número de parâmetros. Por exemplo, um modelo de ~63M parâmetros deve ser treinado em pelo menos ~1,3B *tokens* (nesses casos normalmente arredondamos para 2B).

Sugestões de conjuntos de dados de alta qualidade para o pré-treino:

1) FineWeb-Edu (`HuggingFaceFW/fineweb-edu`): texto *web* filtrado por qualidade educacional;
2) The Pile: *corpus* diversificado de alta qualidade;
3) RedPajama: reprodução *open-source* do *training mix* do LLaMA;
4) SlimPajama: versão deduplicada do RedPajama.

Infraestrutura e tempo de treinamento

O servidor do curso está disponível para o treinamento. Detalhes sobre acesso constam no moodle. Sugere-se:

1) Usar treinamento distribuído (DDP com 2 GPUs) para o pré-treino;
2) Monitorar a *loss* de treino ao longo do tempo e incluir o gráfico na apresentação;
3) Salvar *checkpoints* regularmente para garantir a continuidade do treinamento em caso de falha.

Referência de tempo. O pré-treino de um modelo de ~63M parâmetros treinado em ~2B *tokens* utilizando 2 GPUs do servidor tipicamente leva em torno de 6 horas. Planeje o cronograma do grupo levando em conta filas de espera e eventuais interrupções.

Etapa 2: *mid-training*

Objetivo

O *mid-training* tem como objetivo especializar o modelo a partir do *checkpoint* de pré-treino, expondo-o a dados de natureza conversacional e de raciocínio. Essa etapa utiliza pesos inicializados do pré-treino e continua o treinamento com novos dados e com uma taxa de aprendizado que pode ser diferente em relação ao pré-treino.

Dados

Recomenda-se um *mix* de dados conversacionais e de raciocínio. Sugestões:

1) SmolTalk: conjunto de dados conversacional de alta qualidade (`HuggingFaceTB/smoltalk`);
2) GSM8K: problemas de matemática elementar para capacidade de raciocínio;
3) OpenHermes, UltraChat ou outros conjuntos de dados de instruções de alta qualidade.

O grupo pode escolher um *mix* diferente, desde que justifique a escolha, explicando por que esses dados tornam o modelo mais capaz para as etapas seguintes.

Configuração

Orientações gerais para o *mid-training*:

1) Inicializar a partir do *checkpoint* final do pré-treino (somente pesos do modelo, sem estado do otimizador);
2) Usar uma taxa de aprendizado menor do que a do pré-treino (típico: 1e-4 a 3e-4 com *warmup* curto);
3) Treinar por aproximadamente 1 época sobre o conjunto de dados (ou um número de *tokens* equivalente a ~100M a 300M *tokens*);
4) Formato dos dados: converter todas as amostras para um *template* de *chat* (por exemplo, `User: ...\nAssistant: ...`) e aplicar *sequence packing*.

Etapa 3: *supervised fine-tuning* (SFT)

Objetivo

O SFT tem como objetivo alinhar o modelo ao formato instrucional, ensinando-o a responder perguntas de forma útil e coerente. A principal diferença em relação ao *mid-training* é o uso de uma *loss mask*: somente os *tokens* das respostas do assistente contribuem para o cálculo da *loss*, enquanto os *tokens* do *prompt* (`User:`) são mascarados com `-100`.

Dados e configuração

Para o SFT, recomenda-se:

1) Usar o mesmo conjunto de dados conversacional do *mid-training* (por exemplo, SmolTalk) ou um *subset* curado de alta qualidade;
2) Taxa de aprendizado ainda mais baixa (típico: 1e-5 a 3e-5) e *warmup* curto;
3) Treinar por 1 ou 2 épocas sobre o conjunto de dados de SFT, monitorando o risco de *overfitting*;
4) Inicializar a partir do *checkpoint* final do *mid-training*.

Etapa 4: avaliação

Métricas obrigatórias

Cada grupo deve implementar e reportar, no mínimo, dois tipos de avaliação.

Perplexidade. Calcule a perplexidade (PPL) do modelo sobre um conjunto de validação. A PPL é definida como $\exp(\bar{\ell})$, onde $\bar{\ell}$ é a *cross-entropy* média sobre os *tokens*, e mede quão bem o modelo prevê os *tokens*. Valores menores indicam melhor modelo. A avaliação de perplexidade deve ser feita sobre um *split* de validação do conjunto de dados de pré-treino, ou seja, textos que o modelo não viu durante o treino, e reportada ao longo do treinamento (curva de validação).

***Benchmark* de múltipla escolha.** Além da perplexidade, cada grupo deve implementar e reportar os resultados em pelo menos um dos *benchmarks* abaixo:

1) HellaSwag: completamento de sentenças (raciocínio de senso comum);
2) ARC-Easy ou ARC-Challenge: perguntas de ciências (ensino fundamental e médio);
3) PIQA: raciocínio físico de senso comum;
4) WinoGrande: resolução de anafóricos ambíguos.

Para *benchmarks* de múltipla escolha, utilize a avaliação por *log-likelihood*: para cada alternativa, calcule a probabilidade do modelo gerar aquele texto dado o contexto e escolha a alternativa de maior log-verossimilhança. Isso evita a dependência de geração autorregressiva.

Avaliações adicionais (opcional)

Grupos que quiserem se destacar podem implementar avaliações adicionais, como:

1) Comparação qualitativa de respostas antes e depois do SFT para as mesmas perguntas;
2) Avaliação em GSM8K (acurácia em problemas de matemática elementar);
3) Avaliação humana: o grupo prepara um conjunto de perguntas e membros externos avaliam qual modelo responde melhor.

Demo e apresentação

Demo em modo *chat*

Cada grupo deve desenvolver uma interface de *chat* que permita interagir com o SLM treinado. Uma aplicação Streamlit é suficiente. A demo deve:

1) Carregar o *checkpoint* final do SFT;
2) Permitir enviar mensagens ao modelo e visualizar as respostas;
3) Expor parâmetros de geração ajustáveis (temperatura, *top-k*, número máximo de *tokens*);

4) Ser executada ao vivo durante a apresentação da disciplina.

Apresentação

Cada grupo terá de 10 a 15 minutos de apresentação, incluindo a demo ao vivo. A apresentação deve cobrir:

1) Descrição da arquitetura escolhida e justificativa das decisões de *design*;
2) Análise das *scaling laws* aplicadas: como o número de *tokens* foi determinado?
3) Curvas de *loss* de treino (pré-treino, *mid-training* e SFT);
4) Resultados da avaliação: perplexidade e *benchmark*(s) implementado(s);
5) Discussão: o que funcionou bem? O que mudaria com mais tempo ou mais recursos?
6) Demo ao vivo do modelo em modo *chat*.

Entrega

Entrega única, prazo disponível no moodle. Todos os itens abaixo devem ser entregues via repositório GitHub (*link* no moodle) até a data definida:

1) Repositório GitHub organizado com código de pré-treino, *mid-training*, SFT, avaliação e demo, com README contendo instruções para reproduzir cada etapa;
2) *Checkpoints*: *checkpoint* final do SFT (obrigatório), *checkpoint* do *mid-training* (recomendado) e do pré-treino (recomendado), hospedados no HuggingFace Hub ou no Google Drive;
3) Gráficos das curvas de *loss* de treino das três etapas de treinamento ao longo dos *steps* ou *tokens*;
4) Resultados de avaliação: perplexidade no *split* de validação e resultados no(s) *benchmark*(s) implementado(s);
5) Demonstração de geração de texto, com exemplos de completamento de *prompt* do modelo pré-treinado e de respostas do modelo após o SFT;
6) Slides da apresentação (PDF ou *link* para o Google Slides);
7) Interface de *chat* funcional, com código no repositório e instruções para execução local.

Bons diferenciais, que podem elevar a nota: explorar arquiteturas alternativas além do GPT padrão (por exemplo, GQA, SwiGLU, RMSNorm, Mamba), usar múltiplos *benchmarks*, comparar modelos em diferentes estágios (pós pré-treino *vs.* pós SFT) ou justificar quantitativamente as escolhas de hiperparâmetros via experimentação.

Dicas

Boas práticas

1) Comece pequeno: valide o *pipeline* completo (treino, *checkpoint*, avaliação e demo) com um modelo menor antes de escalar;
2) *Smoke test*: verifique que o modelo consegue *overfittar* um *mini-batch*, o que confirma que o *pipeline* de treino está correto;
3) *Checkpoints* frequentes: treinamentos longos no servidor podem ser interrompidos, portanto retomadas devem ser possíveis;
4) Monitore a *loss* em tempo real: uma *loss* que não decresce no início geralmente indica *bug* no *pipeline* de dados ou na inicialização;
5) *Scripts* separados: separe o código de cada etapa (pré-treino, *mid-training*, SFT e avaliação) em *scripts* distintos, cada um com sua configuração própria.

Referências

1) *Attention Is All You Need*, Vaswani et al. (2017);
2) *Language Models are Few-Shot Learners* (GPT-3), Brown et al. (2020);
3) *Training Compute-Optimal Large Language Models* (Chinchilla), Hoffmann et al. (2022);
4) *LLaMA: Open and Efficient Foundation Language Models*, Touvron et al. (2023);
5) *RoFormer: Enhanced Transformer with Rotary Position Embedding*, Su et al. (2022);
6) SmolLM, HuggingFace (2024), referência de SLM com *pipeline* similar a este projeto;
7) nanoGPT, de Andrej Karpathy, implementação de referência simples e bem documentada (https://github.com/karpathy/nanoGPT).

Regras e considerações

1) **Grupos:** 3 a 5 integrantes;
2) **Uso de código externo:** é permitido usar bibliotecas *open-source* (PyTorch, HuggingFace Datasets, Tiktoken, …) e se inspirar em implementações de referência (por exemplo, nanoGPT). Entretanto, o código do *pipeline* de treino, de avaliação e da demo deve ser escrito pelo grupo. Qualquer código adaptado de terceiros deve ser explicitamente creditado no repositório;
3) **Prazo:** entregas fora do prazo terão desconto de 10% por dia de atraso, até o limite de 50%. Não serão aceitas entregas com mais do que 5 dias de atraso;
4) **Apresentação:** não é obrigatório que todos os integrantes participem da apresentação.